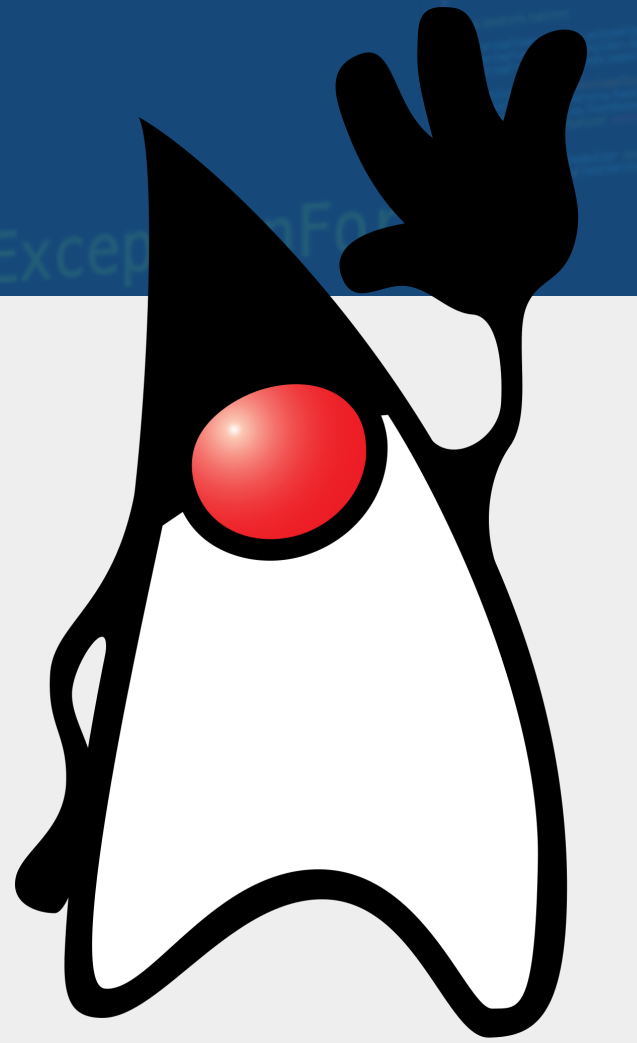


Java Programlama Dili

22. Bölüm Classpath

Akın Kaldıroğlu



Amaç



- Bu bölümde CLASSPATH ele alınacaktır
- Önceki bölümde olduğu gibi bu bölümde de anlatımlar, sınıf üzerinden yapılmasına rağmen anlatılanlar ileride öğreneceğiniz arayüz (interface) vb. Java'daki diğer karmaşık tipler için de geçerlidir



- **Classpath**
- **Classpath Ayarı**
 - *.java ile Derlemede Classpath
 - IDE'lerde Paket ve Classpath
- **JAR ve JMOD**
 - JAR Oluşturma ve Kullanma
- **Ödevler**
- **Bundan Sonra?**

Classpath

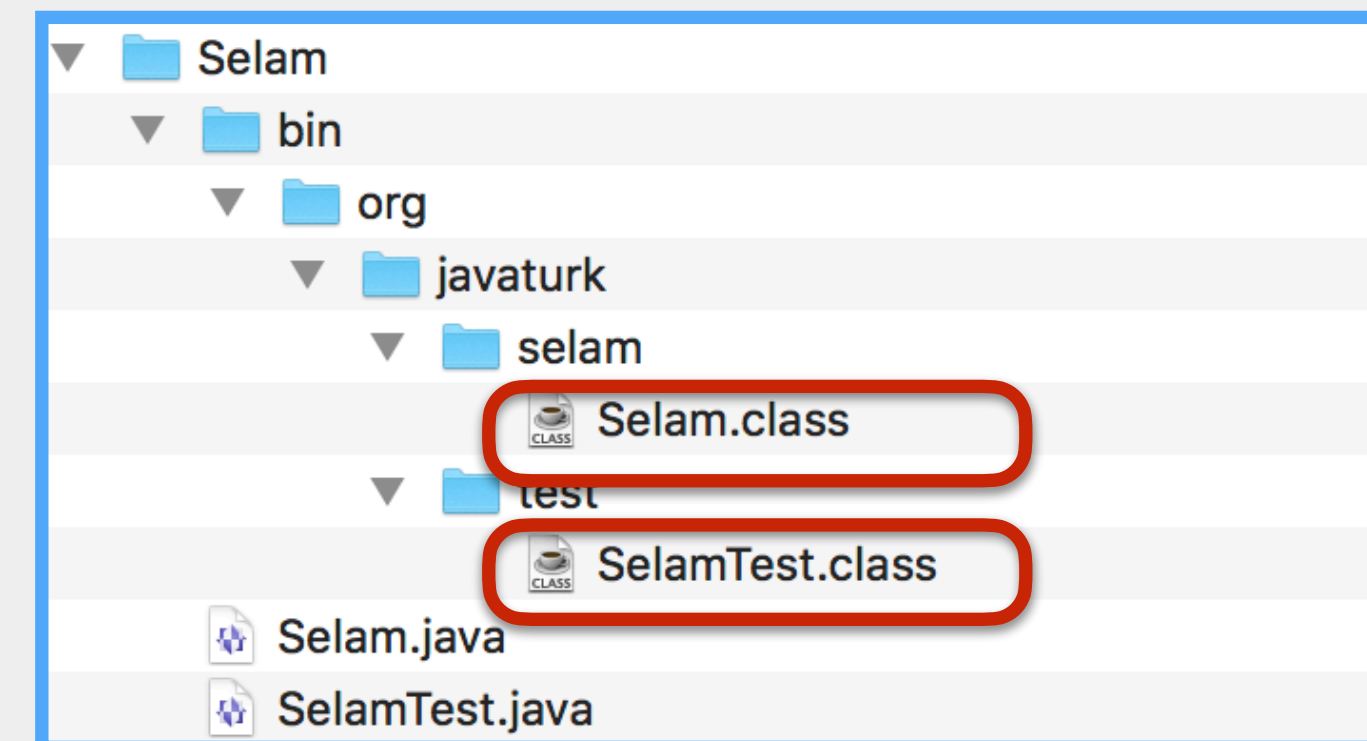


Çalıştırma



- Bir önceki bölümü şöyle bitirmiştik:
- Geriye çalıştırmak kaldı
- Eğer `java org.javaturk.test.SelamTest` komutuyla `SelamTest` sınıfını çalıştırmaya kalkarsak hata alırız

```
.../Selam > java org.javaturk.test.SelamTest
```



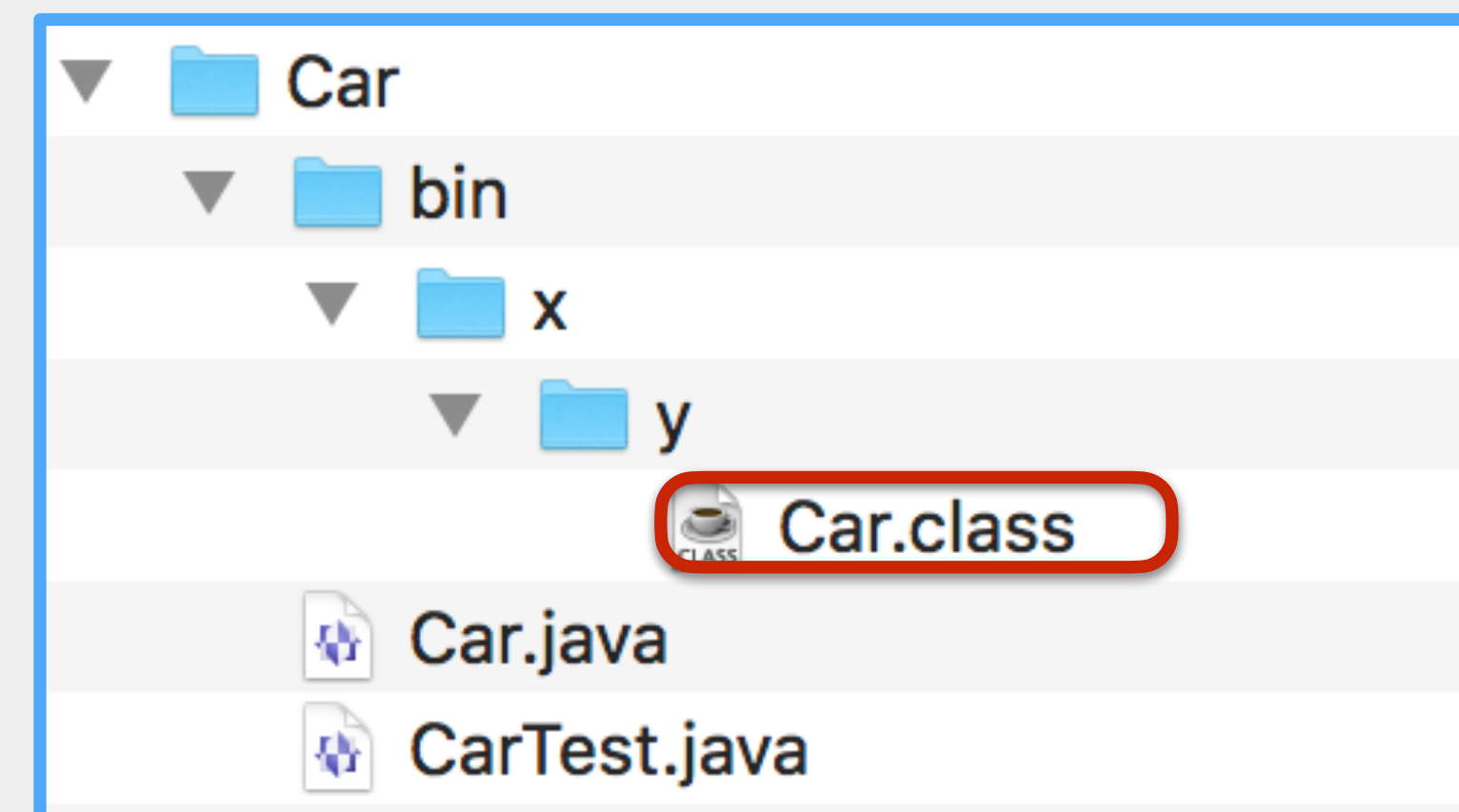
Derleme Hataları - I



- Aslında hata daha derlerken başlar:
- Aşağıdaki örnekte `Car.java` derlenmiş ve paketine uygun dizin yapısında `Car.class` oluşturulmuşken `CarTest.java` derlenmeye çalışıldığında ne olur?

```
package x.y;  
...  
public class Car{...}
```

```
package u.v;  
import x.y.Car;  
...  
public class CarTest{...}
```



```
.../Car > javac -d ./bin CarTest.java
```

Derleme Hataları - II



- `CarTest.java`'da kullanılan `/Users/akin/Desktop/Code/Car/bin/x/y` dizininin altındaki `Car.class` dosyasındaki `Car` sınıfı bulunamadığı için derleme hatası alınır
- Aslen `/Users/akin/Desktop/Code/Car/bin` dizininin altındaki `x.y` paketi dolayısıyla da `x.y.Car` sınıfı bulunamamıştır

Derleme Hataları - III



```
CarTest.java:3: error: package x.y does not exist
import x.y.Car;
      ^
```

```
CarTest.java:17: error: cannot find symbol
    Car car1 = new Car();
      ^
symbol:   class Car
location: class CarTest
```

```
CarTest.java:17: error: cannot find symbol
    Car car1 = new Car();
      ^
symbol:   class Car
location: class CarTest
```

```
CarTest.java:35: error: cannot find symbol
    Car car2 = new Car();
      ^
symbol:   class Car
location: class CarTest
```

```
CarTest.java:35: error: cannot find symbol
    Car car2 = new Car();
      ^
symbol:   class Car
location: class CarTest
```

5 errors

Classpath - I



- Bu bir `classpath` problemidir
- `classpath`, Java'da `class` dosyalarının bulundukları dizinlerin patikalarını (path) gösteren bir değişkendir
- `classpath`'te JVM'in yüklemek için arayacağı sınıfların tam isimlerinin içinde olduğu bir ya da daha çok dizinin patikası sıralanır

Classpath - II



- İşletim sistemlerine göre bu patikalar : ya da ; vb. ayıraçlarla ayrılır
 - Bu ayıraçlara **yol ayırıcı (path separator)** denir
- `classpath`'te de yol ayırıcı ile ayrılarak dizilmiş patikalar vardır

```
C:\Java\bin;C:\workspace\ProjectX\bin;D:\Project\Classes;
```

```
/Users/Java/bin:/Users/akin/workspace/ProjectX/bin:
```

Classpath ve Path



- `classpath`'in, işletim sistemlerinde çalıştırılabilir dosyaların (executables) içinde olduğu dizinleri tutan **PATH** değişkeninden bir farkı yoktur
- `classpath` bir ya da daha çok dizinin patikasını listeler; öyle ki bu dizinlerde JVM'in aradığı Java çalıştırılabilir dosyaları yani `class` dosyaları vardır
- JVM, uygulamanın `class` dosyalarını bulmak için sadece `classpath`'e bakar ve onun gösterdiği patikalardaki dizinleri tarar

Örnek - I



- Daha önce `ClassA` sınıfının paketi `org.javaturk.jp.ch22.packageA` ise örneğin Windows'ta **bir yerde** `org\javaturk\jp\ch22\packageA` şeklinde bir dizin var ve `ClassA.class` dosyası da bu dizindedir demiştik
- O **bir yer** `classpath`'te olmalıdır

Örnek - II



- Eğer `org` dizini `C:\Java\bin`'de ise, `C:\Java\bin`, **classpath**'te olmalıdır

```
C:\Java\bin\org\javaturk\jp\ch22\packageA\ClassA
```

```
CLASSPATH ⇒ C:\Java\bin
```

- Bu durumda `javac` (veya `java`) `classpath`'teki dizinleri tararken `org.javaturk.jp.ch22.packageA` paketine uyan dizin yapısını bulmaya çalışır ve `org` dizini `C:\Java\bin`'de olduğunda, sırayla `org`, altında `javaturk`, onun altında `jp`, ... dizinlerinden giderek `ClassA`'ya ulaşır

CarTest Derlemesi



- Dolayısıyla derleyicinin `CarTest.java`'da kullanılan ve `/Users/akin/Desktop/Code/Car/bin/x/y` dizininin altındaki `Car.class` dosyasındaki `x.y.Car` sınıfını bulabilmesi için `/Users/akin/Desktop/Code/Car/bin` dizini `classpath`'te olmalıdır

```
/Users/akin/Desktop/Code/Car/bin/x/y/Car
```

```
CLASSPATH ⇒ /Users/akin/Desktop/Code/Car/bin
```

Classpath'teki Patikalar



- `classpath`'teki patikaya, sınıfın paketini temsil eden dizin konmaz, sadece sınıfın en üst paketini temsil eden dizine sahip dizinin patikası konur
- Dolayısıyla, sınıfın class dosyasına ulaşabilmek için `classpath`'teki patikaya, sınıfın paketine karşı gelen patika eklenir

```
package x.y;  
...  
public class Car{...}
```

Whole Path to Class File = CLASSPATH + Package Path

/Users/akin/Desktop/Code/Car/bin/x/y/Car

Package

CLASSPATH ⇒ /Users/akin/Desktop/Code/Car/bin

Örnek



- `XClass.class`, `C:\Project\bin\a\b` dizininde, `YClass.class`, `D:\Classes\c\d` dizininde ise `C:\Project\bin` ve `D:\Classes` `classpath`'te olmalıdır
- Aksi takdirde `javac` ve `java`, ne `XClass.class`'i ne de `YClass.class`'i bulabilir

```
package a.b;  
public class XClass{...}
```

```
package c.d;  
public class YClass{...}
```

```
javac -d C:\Project\bin XClass.java
```

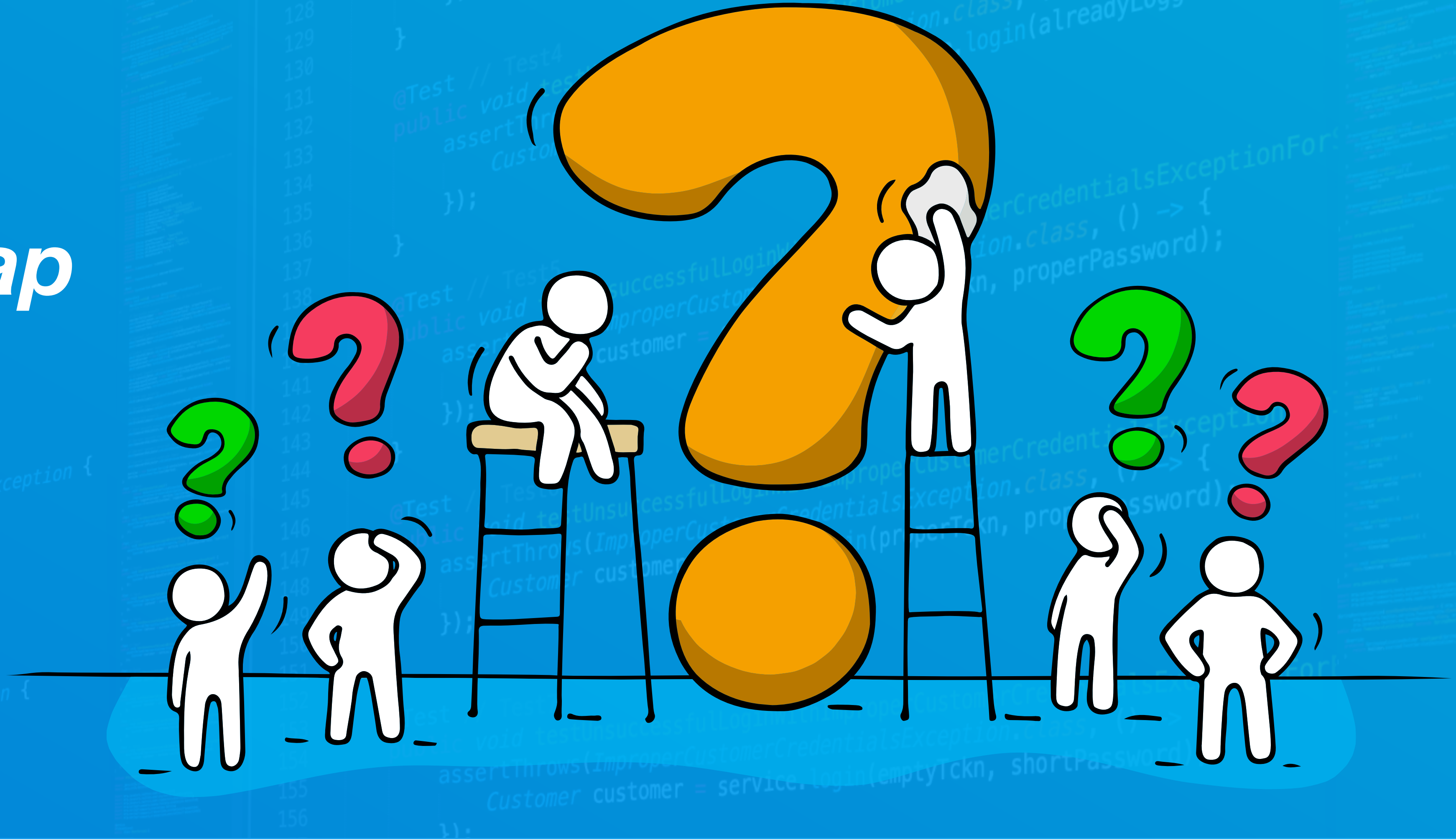
```
javac -d D:\Classes YClass.java
```



```
C:\Project\bin\a\b\XClass & D:\Classes\c\d\YClass
```

```
CLASSPATH ⇒ C:\Project\bin;D:\Classes;
```

Soru ve Cevap Zamanı!



Classpath Ayarı



Classpath Nasıl Ayarlanır?



- Classpath ayarı, `classpath` değişkenine patika ekleyerek yapılır
- Classpath'e 3 şekilde patika eklenebilir:
 - `javac` ve `java` komutlarında `classpath` veya `cp` seçeneğiyle
 - Komut satırında (CMD ya da Terminal/iTerm/Xterm vb.) `CLASSPATH` değişkeni tanımlayarak
 - Sistem değişkenlerine `CLASSPATH` ekleyerek

java/javac Komutlarında Ayarlama



- `classpath` ya da kısaca `cp`, `java` ve `javac` komutlarına verilen bir seçenektir (option, flag) ve `-` ile belirtilir
- Bu kullanımda, her `java` ve `javac` çalışmasında `classpath/cp` seçeneği olmalıdır
- Seçeneklere Java 9 ile aynı işi yapan `--class-path` de eklendi
- Bu şekilde `class` dosyalarını bulamamaktan kaynaklanan bir hata olmaz

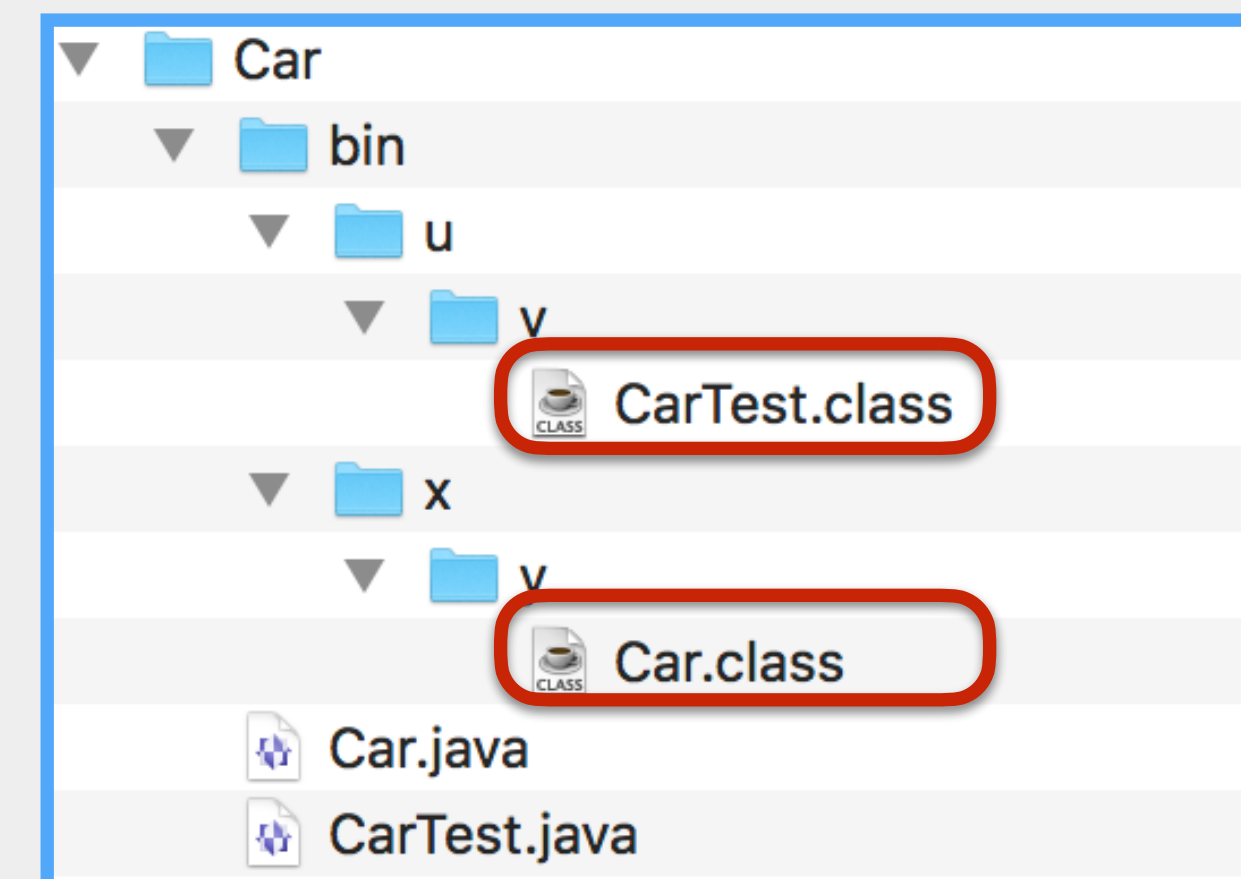
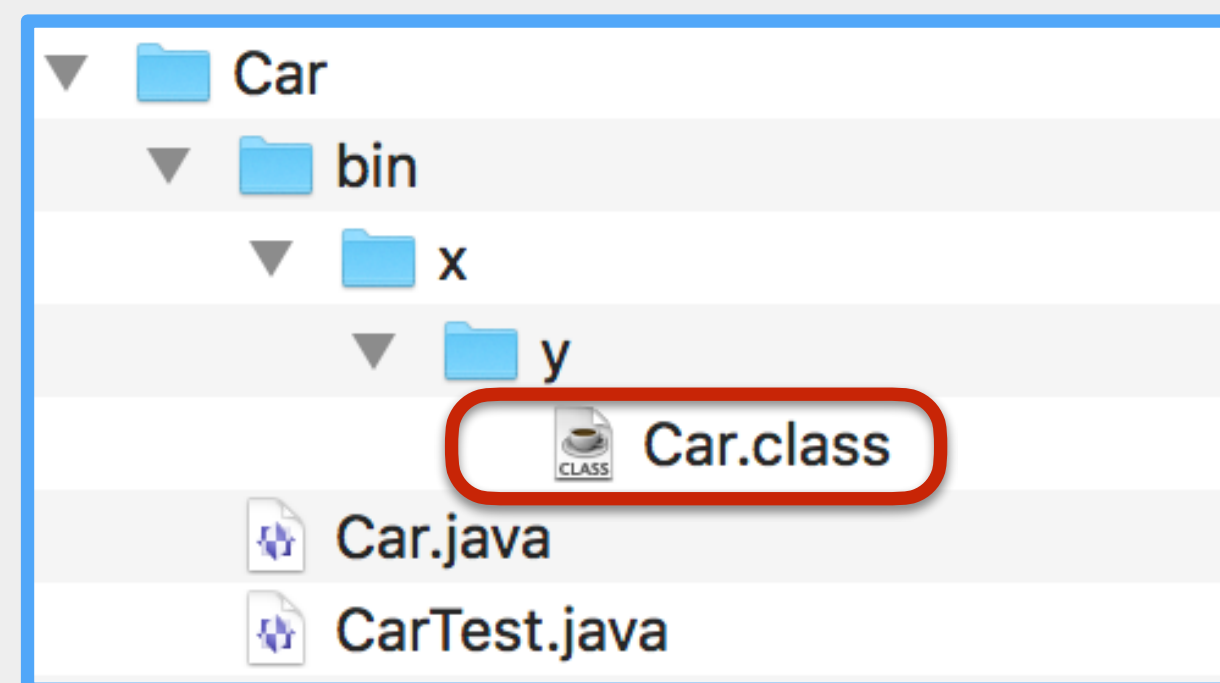
CarTest Derleme Classpath'i



- `CarTest.java` derlenirken alınan hata `classpath` bilgisiyle düzelir
- Bu komut ya `bin`'in olduğu dizinde iken çalıştırılmalı ya da `bin` dizininin tam patikası veya ona ulaşan bir patika verilmelidir

```
.../Car > javac -classpath ./bin -d ./bin CarTest.java
```

```
.../Car > javac -cp ./bin -d ./bin CarTest.java
```



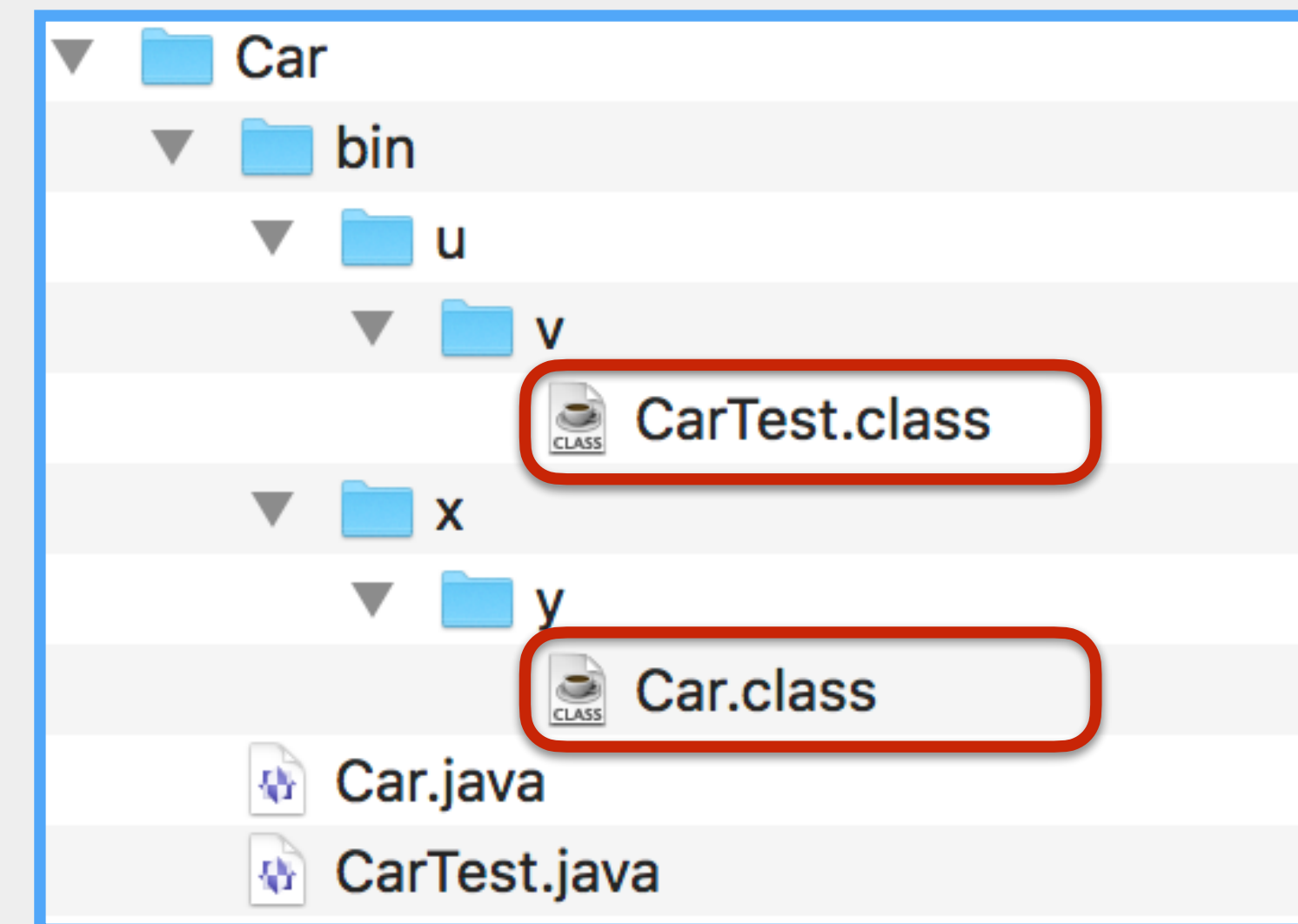
CarTest Çalıştırma Classpath'i - I



- `CarTest`, `java` komutu ile çalıştırılacağı zaman da `u.v.CarTest` ve `x.y.Car` sınıflarının bulunabilmesi için `classpath` seçeneğine ihtiyaç olacaktır

```
.../Car > java -classpath ./bin u.v.CarTest
```

```
.../Car > java -cp ./bin u.v.CarTest
```

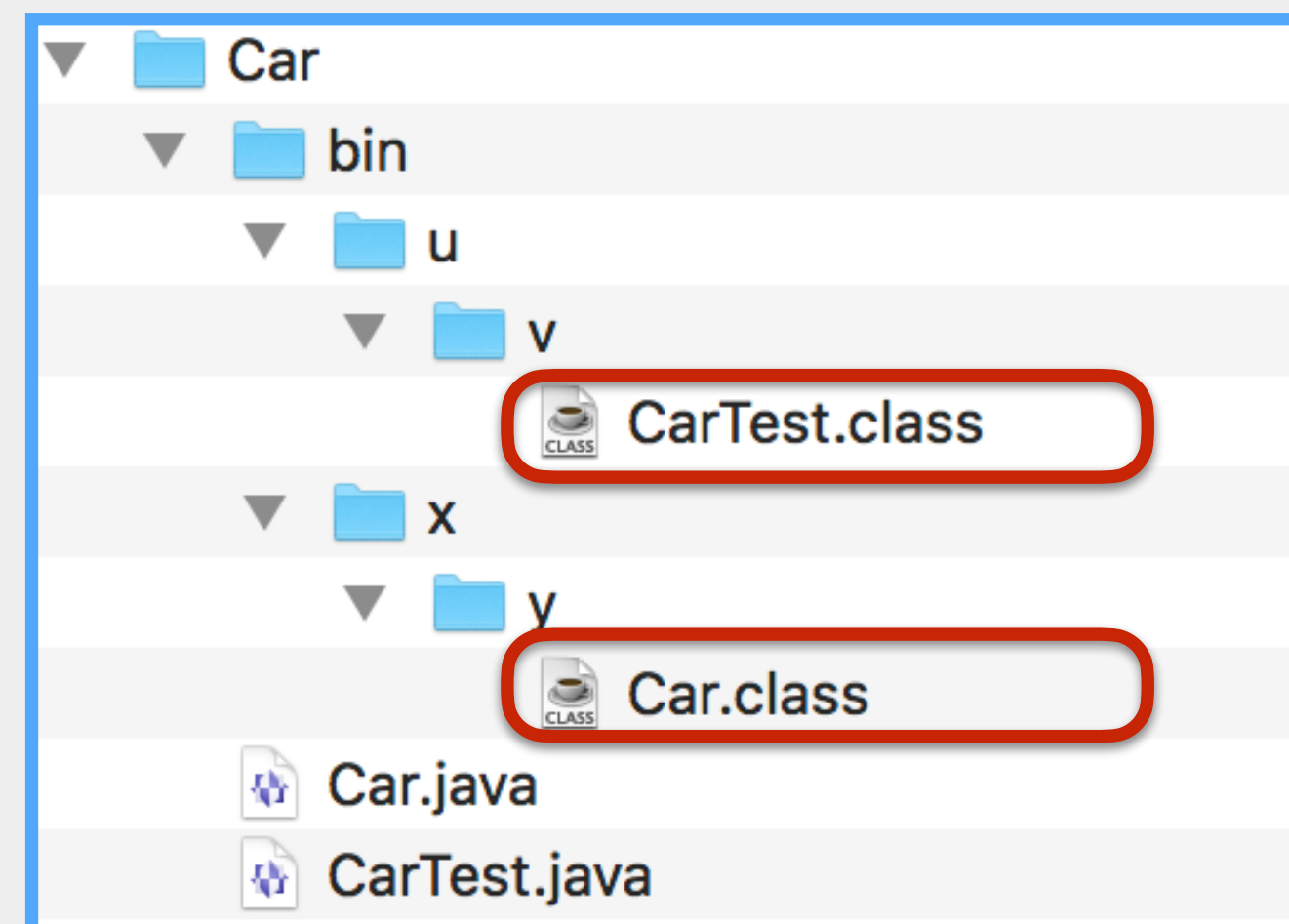


CarTest Çalıştırma Classpath'i - II



- Eğer `java` komutu `bin` dizininin bulunduğu yerde değil de herhangi bir yerde çalıştırılacaksa, bu durumda `classpath` ya da `cp` seçeneğine `bin` dizini bulunacak şekilde *mutlak patika* (*absolute path*) verilmelidir

```
/ > java -classpath /Users/akin/Desktop/Code/Car/bin u.v.CarTest
```

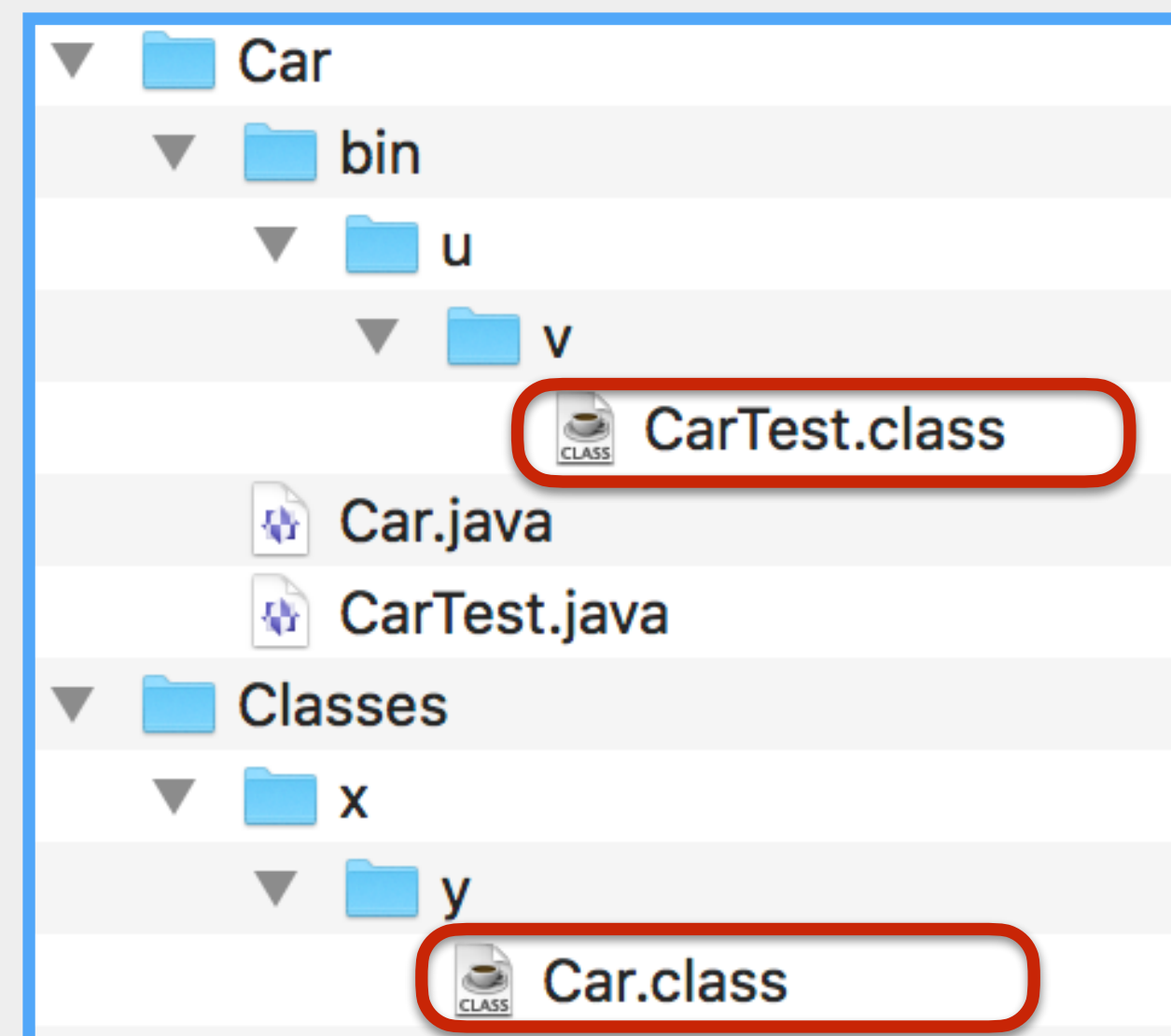


CarTest Çalıştırma Classpath'i - II



- Eğer örneğin `x` ve `u` dizinleri farklı yerlerdeyse bu durumda `java` komutu için `classpath`'e iki tane patika geçmek gerekecektir

```
/ > java -classpath /Users/akin/Desktop/Code/Car/bin:/Users/akin/Desktop/Classes u.v.CarTest
```



Komut Satırında Değişken Olarak - I



- Komut satırında `CLASSPATH`'i ayarlamak, işletim sistemine bağlı bir iştir:

- **Windows**

```
C:\> set CLASSPATH=C:\bin
C:\> echo %CLASSPATH% //Prints the classpath
C:\> java u.v.CarTest
```

- **macOS**

```
/Users/akin > export CLASSPATH=/Users/akin/bin
/Users/akin > echo $CLASSPATH //Prints the classpath
/Users/akin > java u.v.CarTest
```

- **Linux**

```
akin: ~$ export CLASSPATH=/home/akin/bin // bash/ksh
akin: ~$ setenv CLASSPATH /home/akin/bin // csh/tcsh
akin: ~$ echo $CLASSPATH //Prints the classpath
akin: ~$ java u.v.CarTest
```

Komut Satırında Değişken Olarak - II



- Bu yaklaşımın problemi, yapılan ayarın sadece o oturum için geçerli olmasıdır
- Ayarın yapıldığı komut satırı kapatıldığında ayar kaybolur, yeni açılan komut satırında aynı ayarı bir daha yapmak gerekir

Sistem Değişkeni Olarak



- İşletim sistemlerinde, **PATH** gibi farklı verileri tutan pek çok sistem değişkeni olur
- **CLASSPATH** de sistem değişkeni olarak, kullanıcı ya da tüm sistem için kalıcı olacak şekilde ayarlanabilir
- **CLASSPATH**'in sistem değişkeni olarak nasıl ayarlanacağı, Windows, macOS ve Linux için sonraki sayfalarda detaylı anlatılmaktadır

Windows'ta Sistem Değişkeni - I



- Windows'ta CLASSPATH ayarı Ortam Değişkenleri (Environment Variables) ile yapılır
- Ortam Değişkenleri'ne şöyle ulaşılır:

Kontrol Panel (Control Panel) => Sistem ve Güvenlik (System and Security) => Sistem (System) => Gelişmiş Sistem Ayarları (Advanced System Settings) => Ortam Değişkenleri (Environment Variables)

Windows'ta Sistem Değişkeni - II



- Windows'ta **CLASSPATH** şöyle tanımlanır:
- **Ortam Değişkenleri (Environment Variables)** sayfasında **Yeni (New)** tıklanarak açılan pencerede değişken adı olarak **CLASSPATH**, değeri olarak örneğin **C : \bin ; C : \Project\Classes** girilir
- Eğer tanımlı bir **CLASSPATH** varsa değerine yeni dizinler eklenebilir
- Değer olarak girilen patikalar arasına ; koymayı unutmayın

Windows'ta Sistem Değişkeni - II

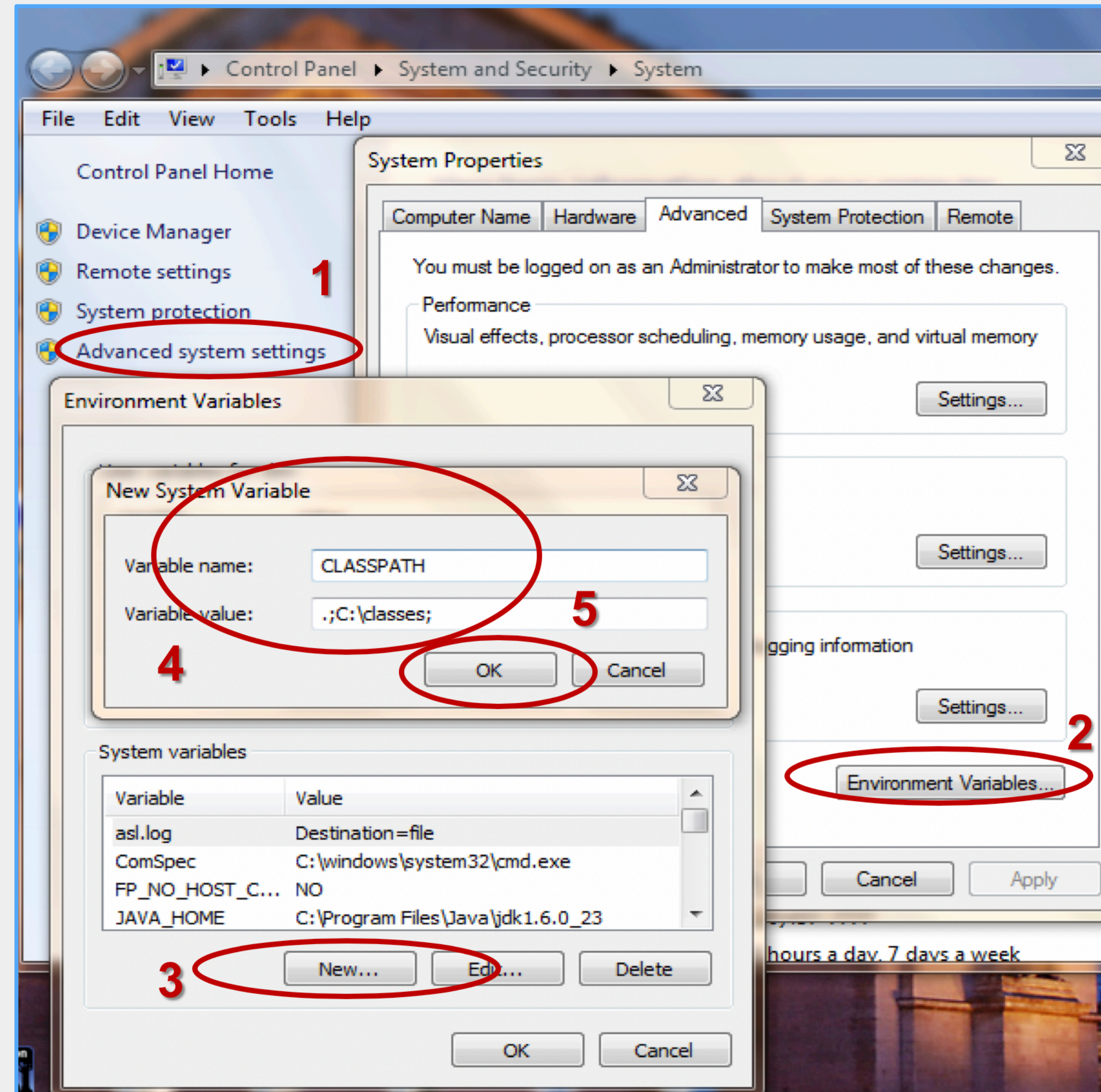


- Komut satırı açık ise, değişikliğin etkisini görmek için kapatılıp yenisinin açılması gereklidir
- Ayrıca komut satırında `set CLASSPATH` ya da `echo %CLASSPATH%` komutları, `CLASSPATH` değişkeninin değerini verir

Windows'ta Sistem Değişkeni - IV



- Windows'ta Control Panel
=> System and Security
=> System ile



macOS'ta Sistem Değişkeni - I



- macOS'ta `CLASSPATH` için kullanıcı dizinindeki `.bash_profile` ya da `.zshrc` dosyasında, *vim* gibi bir metin düzenleyici (editor) kullanarak, patikalar arasına `:` koyarak aşağıdaki gibi tanım yapılması gereklidir:

```
export CLASSPATH=./Users/akin/java/bin:/Users/akin/Desktop/Code/Car/bin
```

macOS'ta Sistem Değişkeni - II



- Terminal açık ise, değişikliğin etkisi için kapatılıp yenisinin açılması ya da terminalde, aynı dizinde `source .bash_profile` ya da `source .zshrc` komutunun çalıştırılması gereklidir
- Terminalde `echo $CLASSPATH` komutu `CLASSPATH` değişkeninin değerini verir

Ubuntu'da Sistem Değişkeni - I



- Ubuntu'da Java ayarları, kullanıcı dizinindeki `.bashrc` dosyasında yapılır
- Bu amaçla komut satırında (terminalde), *vim* gibi bir metin düzenleyici ile `.bashrc` dosyasında değişiklik yapmak gerekir
- Değişikliğin etkisini görmek için açık olan terminalin kapatılıp tekrar açılması ya da terminalde aynı dizinde `source .bashrc` çalıştırılması gereklidir

Ubuntu'da Sistem Değişkeni - II



- Ubuntu'da `CLASSPATH` değişkenini `.bashrc` dosyasında, dizinler arasına : koyarak aşağıdaki gibi tanımlayabilirsiniz:

```
export CLASSPATH=./:/home/ubuntu/bin:/home/ubuntu/Project/classes
```

- Terminalde `echo $CLASSPATH` komutu `CLASSPATH` değişkeninin değerini verecektir

Linux'ta Sistem Değişkeni



- Farklı Linux sistemleri değişik shell yapıları kullanabilir
- Örneğin **cs**h ya da **tcsh** kullanan Linux sistemlerinde **CLASSPATH** değişkeni **.cshrc** ya da **.tcshrc** dosyasında, dizinler arasına : koyarak aşağıdaki gibi tanımlanabilir:

```
setenv CLASSPATH /home/linux/bin
```

- Farklı Linux sistemlerinde kullanım için dokümanlarına başvurun

CLASSPATH'te İçinde Bulunulan Dizin



- Sistemde **CLASSPATH** ayarı yapılmadığı durumda, içinde bulunulan dizin yani `.` daima **CLASSPATH**'tedir
- Ama eğer **CLASSPATH** ayarı yapılmışsa bu durum artık ortadan kalkar
- Bu yüzden **CLASSPATH** ayarı yaparken, kolaylık için daima içinde bulunulan dizini `.` ile **CLASSPATH**'e, hatta en başa ekleyin

Üç Şekilde CLASSPATH Ayarı Varsa



- Eğer `CLASSPATH` ayarı bahsedilen üç şekilde de yapılmışsa, bu durumda `javac/java` komut satırında `-cp` ya da `-classpath` ile verilen değer diğerlerini ezer

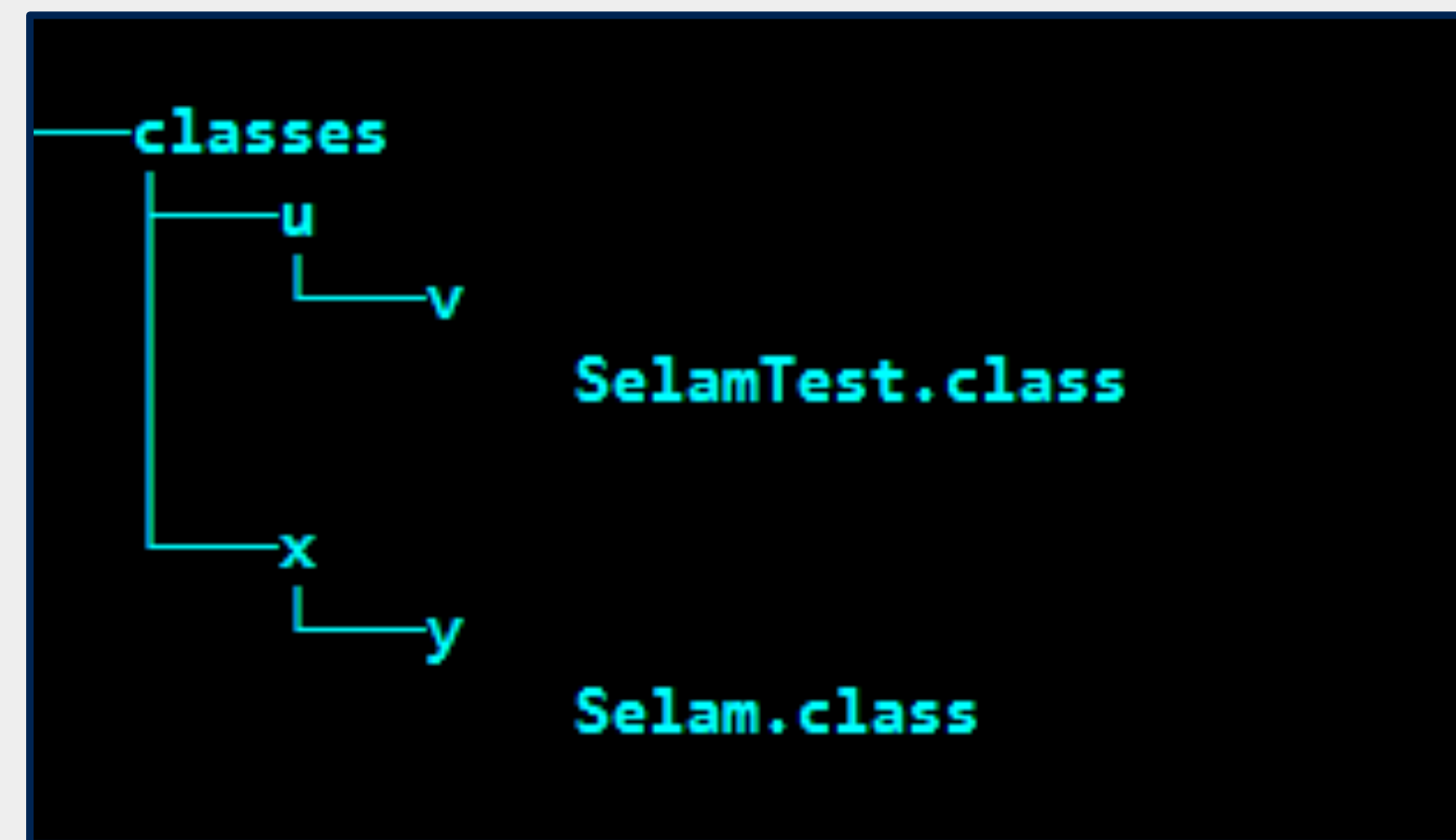


Uygulama

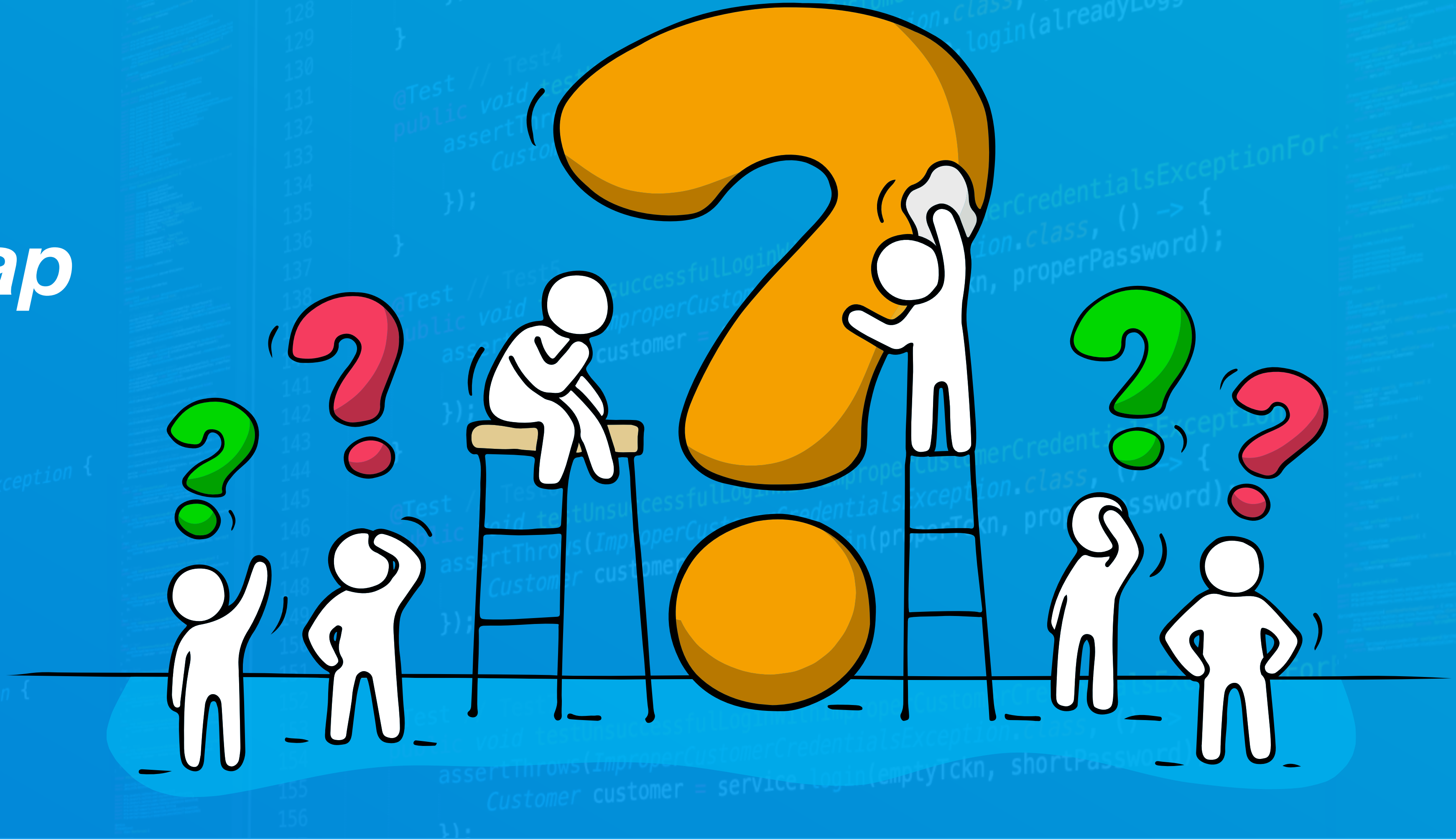
Uygulama



- `Selam` ve `SelamTest` örneği için gerekli CLASSPATH ayarlarını 3 farklı şekilde yaparak `SelamTest.java`'yı derleyin ve çalıştırın



Soru ve Cevap Zamanı!





***.java ile
Derlenmede
Classpath**

*.java ile Derlemede Classpath - I



- Aynı dizindeki birden fazla kaynak dosyası `*.java` kullanarak derlendiğinde classpath problemi yaşanmaz
- `-d` seçeneği kullanılsa da kullanılmasa da sınıflar, kodlarına dahil ettikleri ve beraber derlendikleri diğer sınıfları bulmada problem yaşamazlar
- Classpath problemi, sınıf(lar) derlenirken halihazırda derlenmiş sınıfların `class` dosyalarını bulmada yaşanır

*.java ile Derlemede Classpath - II



- Hatırlayın önce `Car.java` sonra `CarTest.java` derlendiğinde `Car.class` bulunamıyordu
- Ama `*.java` ile beraber derlendiğinde bu problem olmuyordu

```
package x.y;  
public class Car{...}
```

```
package u.v;  
import x.y.Car;  
public class CarTest{...}
```

```
.../Car > javac -d ./bin *.java // OK
```

```
.../Car > javac -d ./bin Car.java // OK  
.../Car > javac -d ./bin CarTest.java // Problem!
```

java ile Çalıştırmada Classpath



- Aynı kolaylık çok dosyalı kaynak kod programlarını doğrudan `java` komutuyla başlatmada da söz konusudur:

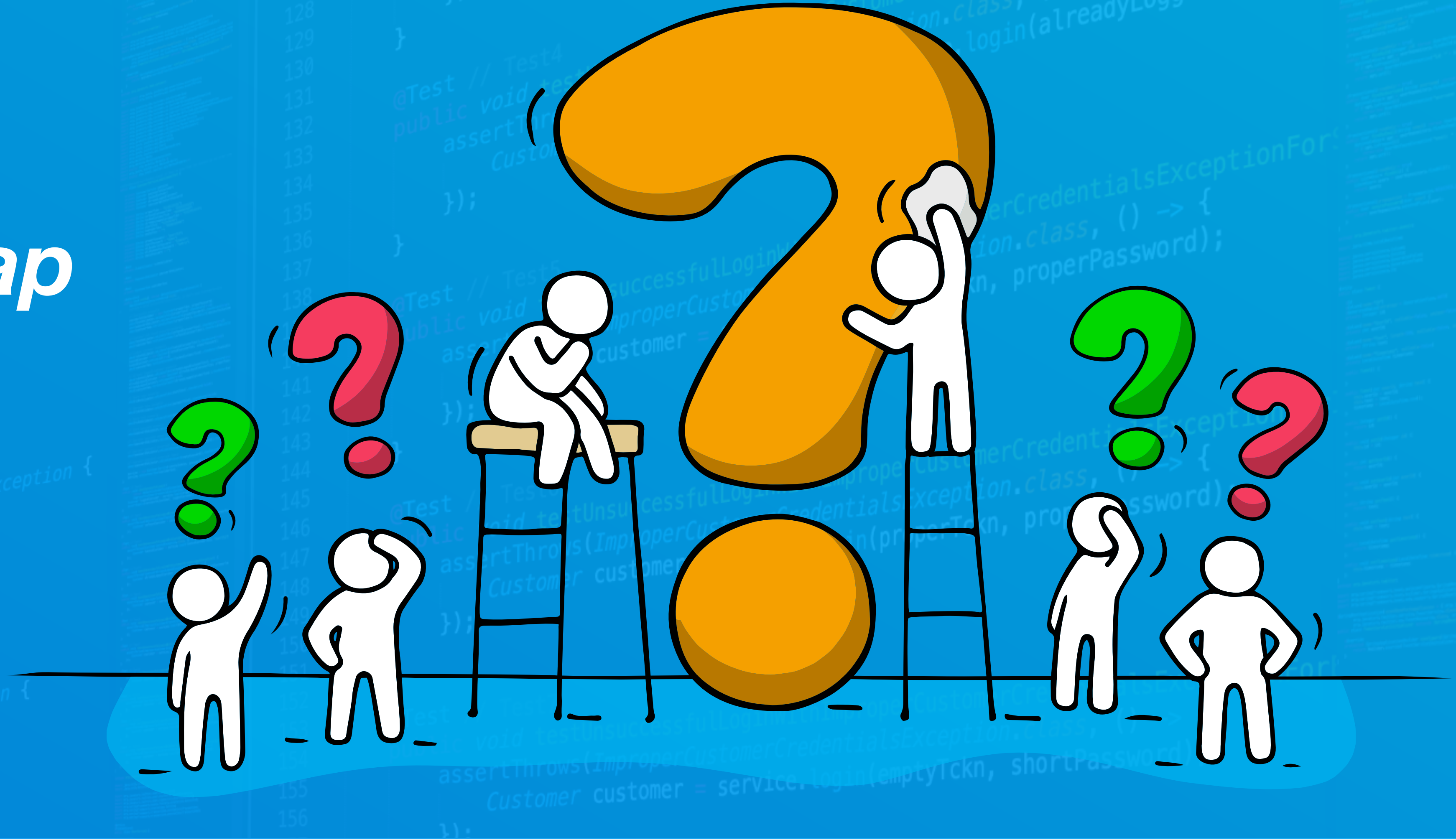
```
package x.y;  
public class Car{...}
```

```
package u.v;  
import x.y.Car;  
public class CarTest{...}
```

```
.../Car > java u/v/CarTest.java // It finds x/y/Car.java
```

- Detaylar için: JEP 458, Launch Multi-File Source-Code Programs

Soru ve Cevap Zamanı!





IDE'lerde Paket ve Classpath

IDE'lerde Proje Yapısı



- IDE'ler, yeni yaratılan bir proje için, workspace'lerindeki proje dizininin altında tipik olarak kaynak kodlar ve derlenmiş class dosyaları için iki dizin yaratır:
- Eclipse, `src` ve `bin`, IntelliJ, `src` ve `out` dizinlerini oluşturur
- `bin` ya da `out` dizininde, sınıfların paketlerine uygun olarak oluşturulmuş dizin yapısı ve dizinlerde `class` dosyaları bulunur
- `src` dizininde, paketle aynı yapıda dizinler ve altlarında `java` kaynak kodları vardır

IDE'de Paket Oluşturmak



- IntelliJ ve Eclipse'te projenin `src` dizini üzerinde sağ tıkla açılan menüden **Package** seçilerek açılan pencerede, paket ismi girilerek yeni bir paket oluşturulabilir
- Var olan paketleri seçerek yine benzer şekilde bu paketler altında yeni paketler oluşturulabilir
- IDE'ler genelde yeni bir sınıf oluştururken de yeni bir paket isminin girilmesine izin verir

IDE'lerde Import Yapmak - I



- IDE'ler, henüz `import` edilmediği için bulamadığı yapılarla ilgili hata verir ve bulunamayan yapının altını kırmızı ile çizer
- IntelliJ, “*Cannot resolve symbol*” mesajı, Eclipse de, “*cannot be resolved to a type*” mesajı verir
- Eğer altı kırmızı ile çizilmiş yapı üzerinde farenin imleci (cursor) bir miktar tutulursa IDE'ler, küçük bir pencerede olması gereken `import` ifadesini gösterir
- Bu ifadeye tıklandığında IDE'ler, gerekli `import` ifadesini koda ekler

IDE'lerde Import Yapmak - II



- Eğer `import` edilecek birden fazla yapı bulunursa, IDE açtığı ufak pencerede bu sınıfların hepsini listeler
- Örneğin kodda `Date` sınıfı kullanılıyorsa açılan küçük pencerede hem `java.sql.Date` hem de `java.util.Date` sınıfları listelenir
- Bu durumda doğru sınıfı seçerek koda dahil etmek önemlidir

IDE'lerde Classpath



- IDE'lerde her projenin ayrı classpath'i vardır
- Projenin classpath'i, proje özelliklerinde görüntülenip değiştirilebilir

IntelliJ'de Proje Özellikleri



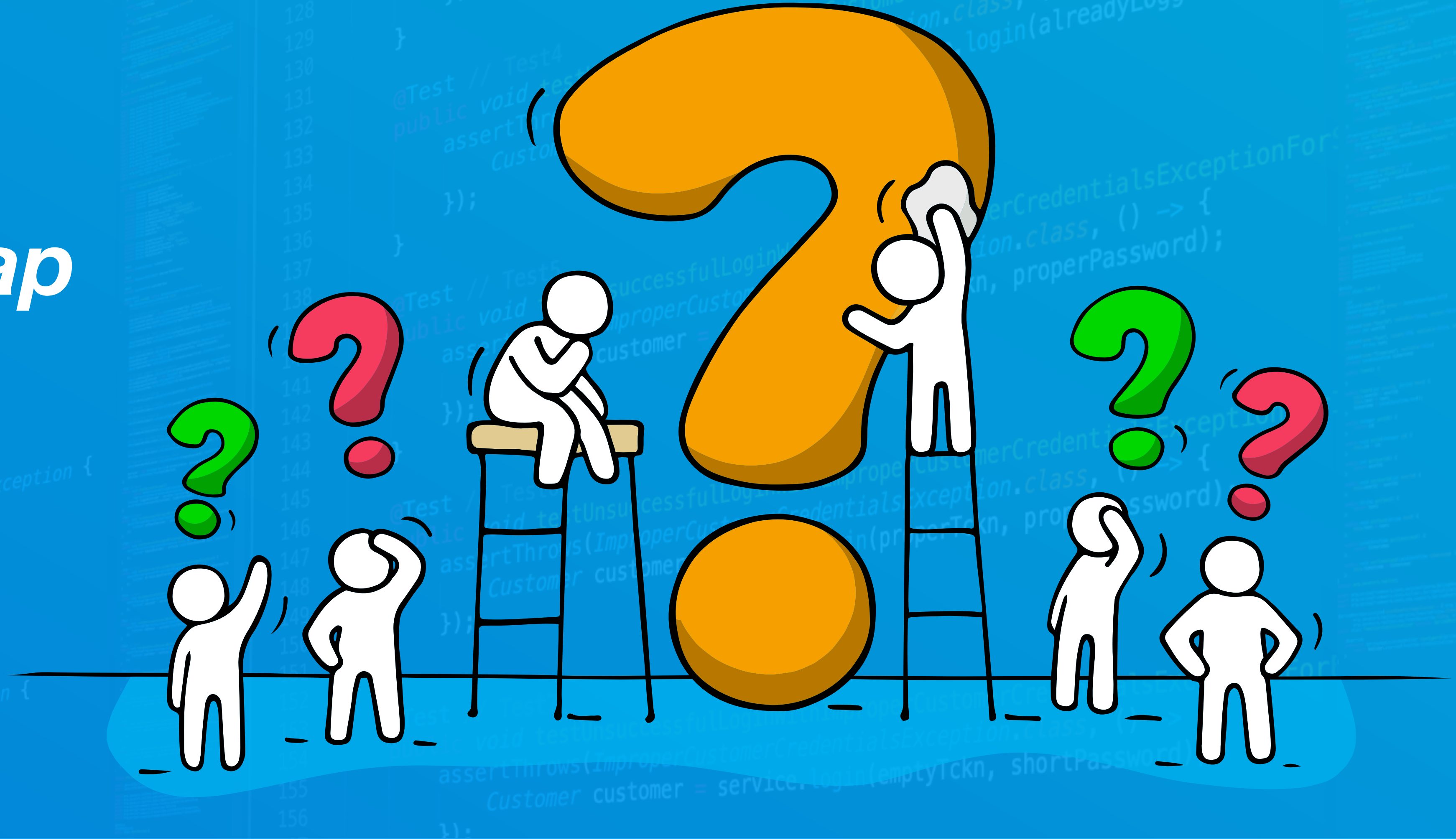
- IntelliJ'de projelerin özelliklerine, proje ismi üzerinde sağ tık yapıp, çıkan menünün en alt tarafındaki **Open Module Settings** seçilerek gidilebilir
- Sonra, **Libraries** seçilip + ve - ile **Classpath**'e ekleme ve çıkarma yapılabilir

Eclipse'te Proje Özellikleri



- Eclipse'te projelerin özelliklerine, proje ismi üzerinde sağ tık yapıp, çıkan menünün en altındaki **Properties** seçilerek gidilebilir
- Sonra **Java Build Path => Libraries** ile **Classpath**'e ulaşılır
- Sonrasında sağdaki düğmeler ile ekleme ve çıkarmalar yapılabilir

Soru ve Cevap Zamanı!



JAR ve JMOD



Java Archive, JAR - I



- **JAR** (**J**ava **A**Rchive), Java'nın arşiv formatıdır
- Bir ya da daha fazla `class` dosyası ve ilgili diğer türden dosyaları bir araya getirip tek bir dosya olarak kullanmanın yolu `jar` oluşturmaktır
- `jar` dosyalarının ismi daima `.jar` ile biter
- Java'nın kütüphane (library) formatı da `jar` dosyasıdır
- Geliştirilen yazılımlar genelde bir ya da daha fazla `jar` dosyası olarak dağıtılır (distribution) ve yaygınlaştırılır (deployment)

Java Archive, JAR - II



- Yazılımlarda kullanılan framework ya da component gibi **üçüncü taraf kütüphaneler (third-party library)** de tipik olarak JAR dosyalarıyla gelir ve projeye ya da yazılıma dahil edilir
- Kütüphanelerin projeye dahil edilmesi, `jar` dosyalarının projenin CLASSPATH'ine eklenmesiyle olur
- `jar` dosyaları CLASSPATH'e eklenmiş kütüphanelere, proje kodlarından ulaşmak için, kütüphanedeki sınıflarla statik üyeleri ya `import` edilir ya da tam ismiyle kullanılır

Java Archive, JAR - II



- Java 9 ile birlikte gelen modül (module) sistemine kadar JDK'nın tüm kütüphaneleri de `jar` dosyaları olarak oluşturulup dağıtılıyordu
- Örneğin JDK 8 kurulumunda *lib* ve *jre/lib* dizinlerinin altında pek çok JAR dosyası vardır ve bu JAR dosyaları `javac` ve `java` komutları tarafından her zaman bulunur



- Java'ya 9. sürüm ile birlikte modül (module) sistemi gelmiştir
 - Modül, paketten daha yukarıda bir soyutlama yapısıdır
 - Modül, birbirleriyle ilgili paketlerden oluşan bir bileşik yapıdır
- Modül sistemi, JAR gibi zip tabanlı ama farklı bir dosya formatı getirdi: **jmod**
- Ayrıca JDK'nın *bin* dizininde *jmod* dosyası olarak oluşturulan Java modüllerini yönetmek üzere, **jmod** isimli bir komut da vardır

Jmod - I



- JDK 9 ve sonrasında JDK'nın kütüphaneleri de artık, *jmod* dosyaları olarak gelmektedir
- Nitekim JDK'nın 9 ve sonrasında kurulumlarında *jmods* dizininde pek çok *jmod* dosyası vardır
- *jmod* komutu ile *jmod* dosyaları oluşturabilir ve var olanlar açılabilir (extract)

Jmod - II



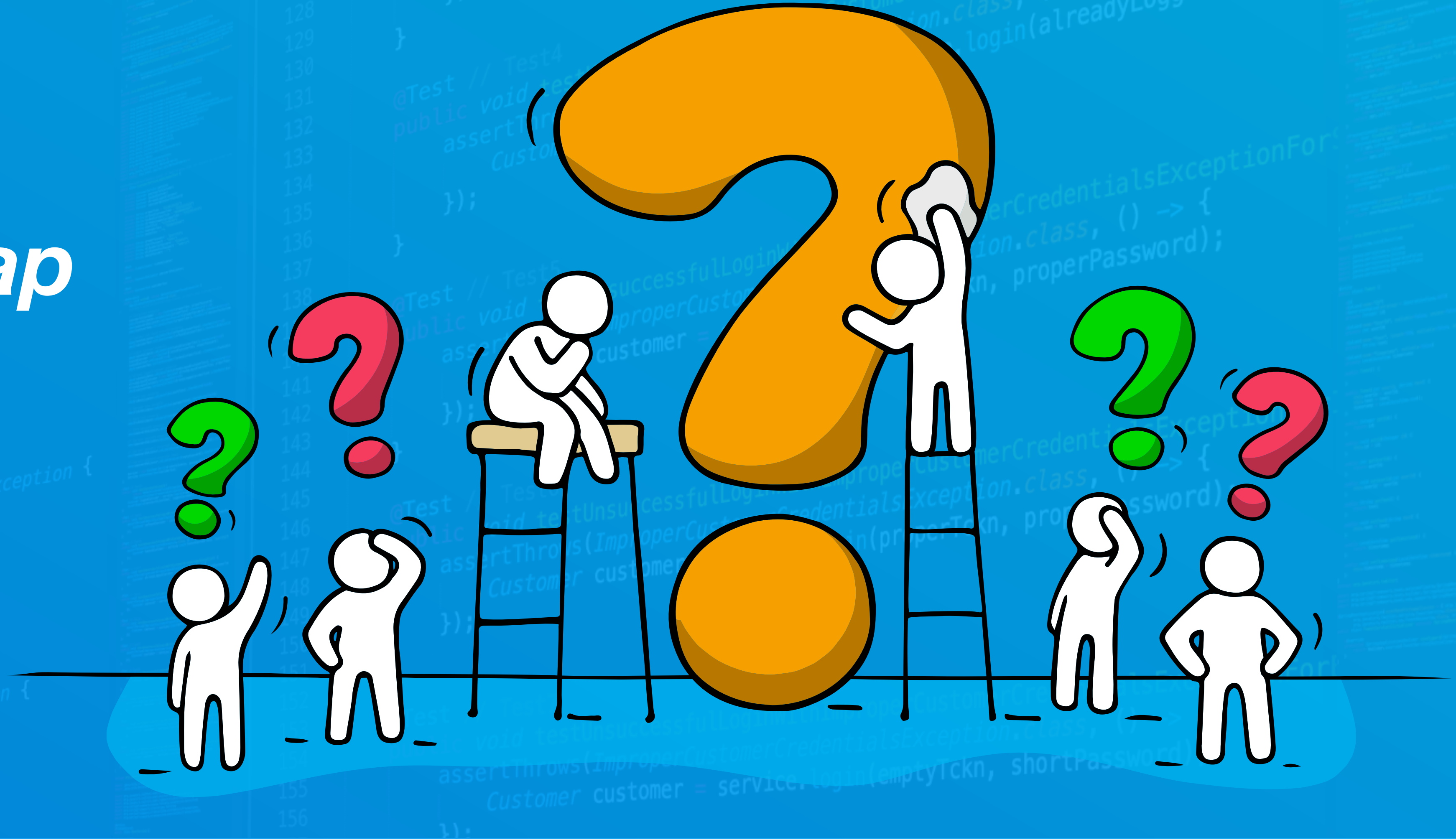
- JDK 9 ve sonraki kurulumlardaki bir *jmod* dosyası, *jmod* komutu ile açıldığında, minimal bir JRE için gerekli tüm kütüphane, konfigürasyon, vb. dosyaların, `class` vb. dosyalar olarak bir araya getirildiği görülebilir

JAR ve JMOD



- Üçüncü taraf kütüphane vb. yapılar jar ya da jmod dosyası olarak oluşturulup dağıtılabilir
- Biz bu eğitimde modül yapısını ele almayacağımız için jmod ile ilgili incelemeye devam etmeyeceğiz
- Dolayısıyla dağıtmak ve kullanmak amacıyla `jar` dosyalarının nasıl oluşturulduğunu ve projelere dahil edildiğini inceleyeceğiz

Soru ve Cevap Zamanı!





JAR Oluşturma ve Kullanma

jar Komutu



- JAR formatında arşiv oluşturan komutun ismi `jar`'dır ve diğer Java araçları gibi JDK kurulumundaki `bin` klasöründedir
- JAR dosyaları aynı zamanda çalıştırılabilen (executable) Java dosyaları olarak da kurgulanabilir
- Bu durumda `jar` dosyaları komut satırından ya da uygulama sunucusu vb. ortamlardan çalıştırılabilir

Jar Dosyası Oluşturma



- Jar dosyası oluşturmak için gerekli bilgileri `jar` komutundan öğrenebilirsiniz

```
jar --help
```

- Aşağıdaki komutla `Selam.jar` isimli bir jar dosyası oluşturup *bin* dizininin altındaki her şeyi bu dosyaya koyabilirsiniz

```
jar cvf Selam.jar -C bin/ .
```

Jar Dosyasını Classpath'e Ekleme



- Aşağıdaki komutla içinde bulunulan dizindeki `Selam.jar` dosyasını `classpath`'e koyarak önce `javac` ile derleyip sonra `java` ile çalıştırabilirsiniz:

```
javac -classpath ./Selam.jar -d ./bin SelamTest.java
```

```
java -classpath ./Selam.jar:./bin org.javaturk.test.SelamTest
```

Jar Dosyasını Çalıştırma - I



- Çalıştırılabilir jar dosyası için, jar dosyası yaratılırken içinde **main** metodun olduğu sınıfın belirtilmesi gerekir
- Bu amaçla jar dosyasına, içinde bu bilginin yer aldığı bir *manifest* dosyasının eklenmesi gerekir ya da **main** metodun olduğu sınıf aşağıdaki şekillerde belirtilir:

```
jar cvfe Selam.jar org.javaturk.test.SelamTest -C bin/ .
```

```
jar --create --file Selam.jar --main-class org.javaturk.test.SelamTest -C bin .
```

Jar Dosyasını Çalıştırma - II



- Oluşan `Selam.jar` dosyası aşağıdaki gibi `java -jar` komutuyla çalıştırılabilir:

```
java -jar Selam.jar
```

IntelliJ'de jar Dosyası Yaratma



- IntelliJ'de bir projenin jar dosyası iki adımda şöyle oluşturulur:
- `File => Project Structure => Artifacts => +`, jar dosyasının içeriği belirlenir
- Sonrasında `Build => Build Artifacts` ile ilgili artifact için bir jar dosyası oluşturulur

Eclipse'te jar Dosyası Yaratma



- Eclipse'te jar dosyası şöyle oluşturulur:
 - Proje üzerinde sağ tık, **Export** => **General** => **Archive File** ile gerekli bilgiler girilip projenin jar dosyası yaratılabilir

IntelliJ ve Eclipse'te Classpath



- IntelliJ ve Eclipse'te bir projenin `classpath`'ine jar dosyası eklemek için, daha önce `class` dosyası eklemek için belirtilen yöntem kullanılır

Selam ve SelamTest



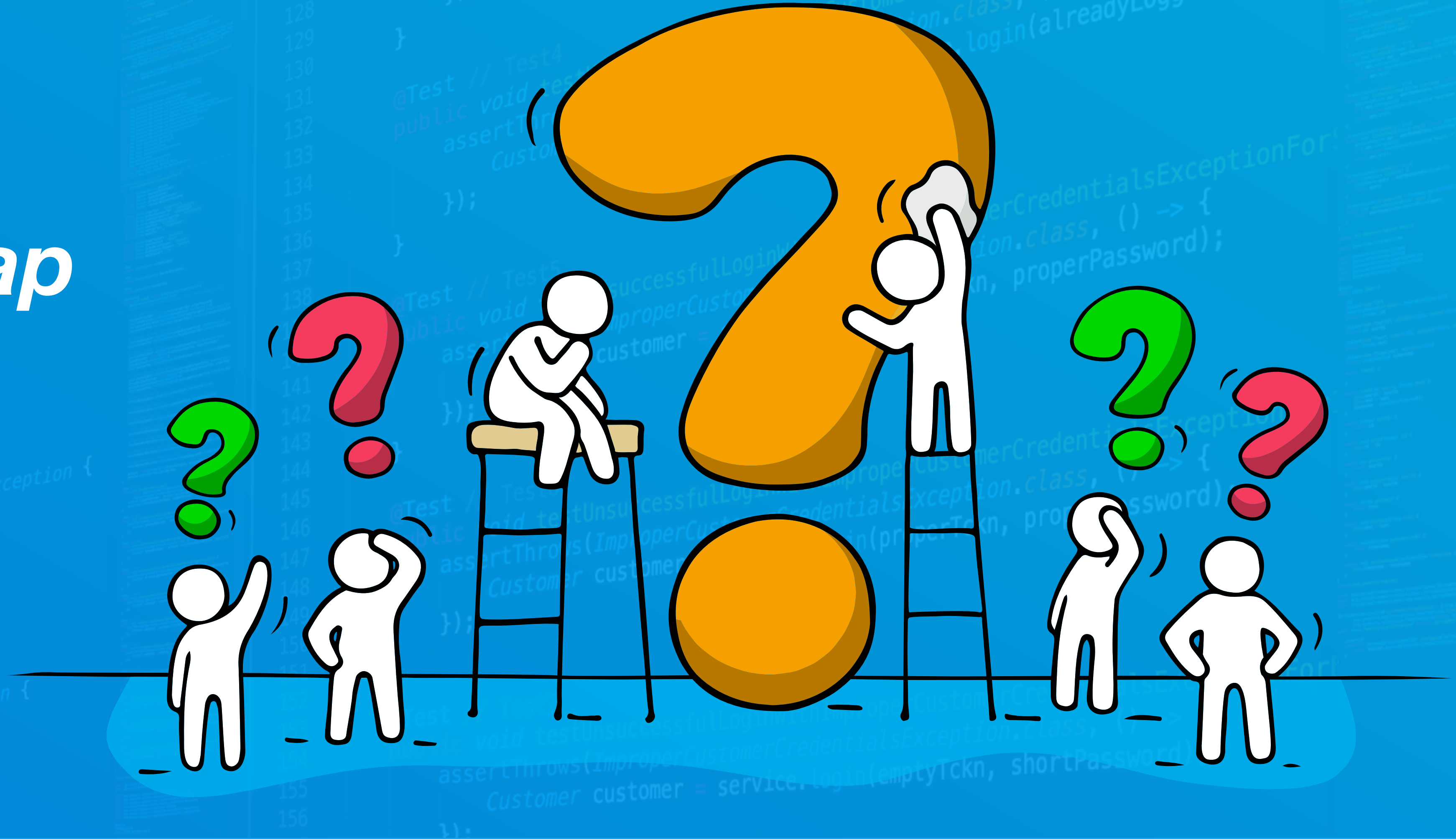
- `Selam.class` sınıfını kullanarak `Selam.jar` isimli bir jar dosyası oluşturun
- Sonrasında hem komut satırında hem de IDE ile bu dosyayı classpath'e koyarak `SelamTest` sınıfını çalıştırın

Selam ve SelamTest



- `Selam.class` sınıfını kullanarak `Selam.jar` isimli bir jar dosyası oluşturun
- Aynı jar dosyasını çalıştırılabilir (executable) olarak `SelamTest` sınıfını da içerecek şekilde oluşturun
- Sonrasında bu dosyayı komut satırında `java -jar` komutu ile çalıştırın

Soru ve Cevap Zamanı!



Ödevler

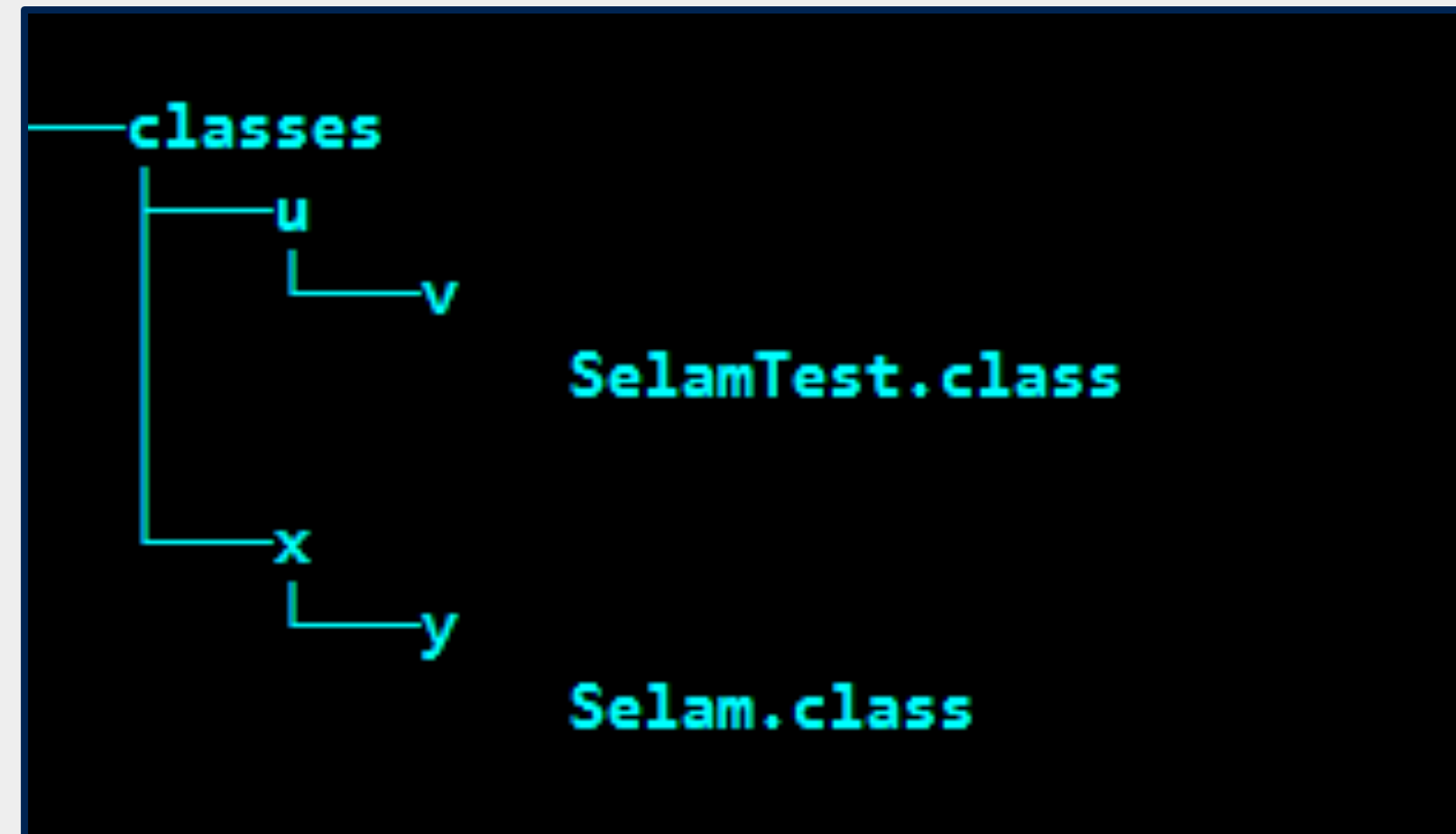




1. Bu dersin projesinin kullandığınız IDE'deki, VS Code, Eclipse veya IntelliJ'in workspace'indeki dizinine gidip, **src** ve **bin** veya **out** dizinlerini inceleyin
 - **out/bin** dizinindeki paketleri ve içlerindeki **class** dosyalarını inceleyin, **class** dosyalarının kendisinden üretildikleri Java kaynak kodlarındaki **package** cümlesine uygun olarak dizinlerin altına konduğunu gözlemleyin



2. IDE'de `SelamTestProject` isimli yeni bir proje oluşturun
 - a. Projede sadece `SelamTest.java` sınıfını daha önceki gibi, aşağıdaki pakete sahip olacak şekilde oluşturun





- b. Bu projede `Selam.java` dolayısıyla `Selam.class` olmamalı, bu dosyalar daha önce IDE kullanmadan yaptığınız uygulamada olduğu yerlerinde kalmalıdır
- c. Projede sadece `SelamTest.java` olacaktır
- d. IDE'yi kullanarak projenin classpath'ini `Selam.class` sınıfını `import` edip kullanabilecek şekilde değiştirin ve `SelamTest`'i çalıştırın



3. Bu bölümün homework paketindeki `StringOperations.java` kaynak kodunu derleyip çalıştırın
 - a. Bu amaçla https://commons.apache.org/lang/download_lang.cgi adresinden Apache Commons Lang'in jar dosyasını indirip projenin classpath'ine ekleyin
 - b. Derleme ve çalıştırmayı komut satırında yapın
 - c. Bu örnekte kullanılan `StringUtils` sınıfının API'si için: <https://commons.apache.org/proper/commons-lang/apidocs/org/apache/commons/lang3/StringUtils.html>

Bundan Sonra?



Bundan Sonra?



- Bundan sonra sarmalama (encapsulation) ve bilgi saklamayı (information hiding) ele alacağız

Bölüm Sonu

Soru ve Cevap Zamanı!

